Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT

ON

LEARNING MANAGEMENT SYSTEM

Submitted To

Department of Computer Application

Ed-Mark College

In partial fulfillments of the requirements for the Bachelor's degree in Computer Application

Submitted By:

Ankisa Ghalan

[TU Reg. No: 6-2-727-6-2022]

April , 2026

Under the Supervision of

Mr. Shyam Maharjan

Tribhuvan University

Faculty of Humanities and Social Sciences

Ed-Mark College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Ankisa Ghalan** entitled "**Learning Management System**" in partial fulfillment of the requirements for the degree of Computer Application is recommended for the final evaluation.

Project Supervisor

Mr. Shyam Maharjan

Department of Bachelor in Computer Application Ed-Mark College

Kalanki, Kathmandu

Tribhuvan University

Faculty of Humanities and Social Science

Ed-Mark College

LETTER OF APPROVAL

This is to certify that this project is prepared by **Ankisa Ghalan** entitled "**Learning Management System**" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

--- **Mr. Shyam Maharjan** **Supervisor** Department of bachelors in Computer Application Ed-Mark College Kalanki, Kathmandu	--- **Er. Sanjay Kumar Sah** **Co-Ordinator** Department of bachelors in Computer Application Ed-Mark College Kalanki, Kathmandu
--- **Internal Examiner**	--- **External Examiner**

ACKNOWLEDGEMENT

It is with great pleasure that I present this project report on **"Learning Management System"**. The successful completion of this work would not have been possible without the guidance, encouragement, and support of several individuals and institutions, to whom I express my heartfelt gratitude.

First and foremost, I extend my sincere appreciation to **Tribhuvan University** for providing me with the opportunity to undertake this project. I owe a special debt of gratitude to my **Project Supervisor Mr. Shyam Maharjan** and **Program Coordinator Er. Sanjay Kumar Sah**, whose invaluable guidance, insightful advice, and unwavering support were instrumental in shaping this project. Their mentorship has been truly inspiring.

I am also deeply grateful to **Ed-Mark College** and my esteemed **Principal Mr. Shiva Prasad Adhikari**, for fostering an environment that nurtures academic excellence and personal growth.

I also extend my appreciation to my teammates for their dedication, collaboration, and perseverance, which made this endeavor possible. A heartfelt thank you to my seniors for their valuable insights and to my friends for their constant encouragement and assistance throughout this journey.

Lastly, I am profoundly grateful to my families for their unwavering support, motivation, and belief in me. Their encouragement has been my greatest source of strength.

I hope this report meets the academic standards and serves as a meaningful contribution to our field of study.

Thank you all for being a part of this incredible journey.

Sincerely,

Ankisa Ghalan

[Reg. No: 6-2-727-6-2022]

6th Semester

ABSTRACT

The Learning Management System (LMS) is designed to provide an efficient and centralized platform for delivering, managing, and tracking online educational content. The system enables educators to create courses, upload learning materials, organize lessons, and evaluate student performance through quizzes and assignments. Students can easily enroll in courses, access multimedia resources, submit coursework, and monitor their learning progress in real time. To enhance interaction, the system incorporates features such as user authentication, course tracking, progress analytics, and communication tools. On the administrative side, it provides capabilities for managing users, monitoring course activities, and maintaining the overall learning environment. By leveraging modern web technologies and database systems, the LMS enhances teaching efficiency, improves accessibility, and transforms traditional classroom learning into a flexible digital experience.

Keywords: ***Learning Management System, E-learning, Online Education, Course Management, Student Enrollment.***

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

LMS	Learning Management System
CASE	Computer Aided Software Engineering
CSS	Cascading Style Sheet
DFD	Data Flow Diagram
ER-Diagram	Entity Relationship Diagram
HTML	Hyper Text Markup Language
Js	JavaScript
UI	User Interface
NFRs	Non-Functional Requirements
FRs	Functional Requirements

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Introduction

A Learning Management System (LMS) is a web-based software application designed to support the creation, management, and delivery of educational courses and training programs. In this project, the LMS is developed using React.js for the frontend to provide a dynamic and responsive user interface, and Node.js with a backend server and database to handle data processing, authentication, and storage.

The system offers a centralized platform where students, educators, and admin can interact efficiently. Students can register and log in securely, browse available courses, access learning materials such as videos and notes, attend online or recorded classes, and track their learning progress. Educators can create and manage courses, upload content, evaluate student performance, and provide feedback through the system. Admin is responsible for managing users (students, educators), courses, and overall system operations.

The LMS supports both synchronous learning (live classes and real-time interactions) and asynchronous learning (recorded lectures and self-paced study), enabling flexible learning anytime and anywhere. Key features of the system include user authentication, role-based access control, course and content management, progress tracking, and assessment handling. By automating academic processes and reducing manual effort, the LMS improves learning efficiency and simplifies administrative tasks for educational institutions.

1.2 Problem Statement

In traditional learning environments, managing courses, distributing study materials, tracking student progress, and conducting assessments is often time-consuming and inefficient. Students face difficulties accessing resources remotely, while educators struggle with organizing classes, monitoring performance, and providing timely feedback. Institutions also lack a centralized system to streamline communication, automate administrative tasks, and ensure effective learning outcomes.

There is a need for a Learning Management System (LMS) that provides a digital platform to overcome these challenges by offering easy access to learning resources, effective course management, performance tracking, and interactive learning opportunities.

Some of the problem statement in this learning management system are as follows;

a. Inefficient Course Management

Traditional methods of organizing courses, scheduling classes, and managing academic content are time-consuming. Educators spend significant effort handling manual paperwork and structuring course materials.

b. Difficulty Distributing Study Materials

Students often struggle to access study resources, especially when learning remotely. Physical distribution or scattered digital materials create delays and inconsistencies in learning.

c. Lack of Centralized Student Progress Tracking

Tracking student performance manually is inefficient. Educators find it difficult to monitor progress, identify learning gaps, and provide timely interventions without a centralized tracking system.

d. Limited Accessibility to Learning Resources

Students cannot easily access notes, assignments, and lectures outside the classroom. This restricts self-paced learning and creates barriers for learners who need flexible access.

e. Poor Communication Between Students and Educators

Traditional learning environments lack a unified system for announcements, discussions, and updates. This leads to miscommunication and delays in sharing important information.

f. Lack of a Centralized Digital Learning Platform

There is no single integrated platform that combines course management, resource sharing, communication tools, and progress monitoring, resulting in a fragmented learning experience.

1.3Objective

- To create online platform for managing courses, learning materials, and user data.
- To enable students to access study resources, and assessments anytime and anywhere.
- To develop tools for admin to manage students, educators and other activities efficiently.

1.4Scope and Limitations

a. Scope

- **User Authentication and Authorization**
  The system allows users to register and log in securely. Role-based access is provided for administrators, educators, and students to ensure that each user can only access permitted features.
- **Admin Dashboard Management**
  Admin can manage users, courses, categories, and system settings through a centralized admin dashboard. This ensures smooth control and monitoring of LMS activities.
- **Course Creation and Management**
  Educators can create, edit, and delete courses, upload learning materials, and organize lessons according to course structure.
- **Student Enrollment and Course Access**
  Students can enroll in available courses and access course materials such as videos, documents, and assignments.
- **Responsive User Interface**
  The frontend developed using React provides a responsive and user-friendly interface that works across desktops, tablets, and mobile devices.
- **Progress Tracking and Monitoring**
  The system tracks student progress based on completed lessons or assessments, helping learners and educators monitor performance.
- **Database Management**
  The system stores user data, course details, enrollment information, and progress records securely in a database connected to the Node.js server.

b. Limitations

- It may lack advanced encryption or data protection measures.
- It does not include real-time communication features.
- The system does not fully support offline access to learning materials.

1.5. Report Organization

This report is organized into 5 chapters:

Chapter 1 – Introduction: This chapter introduces the problem statement, objectives and limitations of the project.

Chapter 2 - Requirement and Feasibility Analysis: This chapter describes about the functional and non-functional requirements, economic feasibility, technical feasibility, operational feasibility and scheduling feasibility.

Chapter 3 - System Design: This chapter introduces about the system and interface design of the project app.

Chapter 4 - Implementation and Testing: This chapter clearly illustrates the methods and tools used to implement the project.

Chapter 5 - Conclusion and Future Works: This is the final chapter that concludes the project and talks about our future plans with the project.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1. Background Study

A Learning Management Systems (LMS) play a crucial role in enhancing the educational experience by providing a structured platform for teaching and learning. They allow students to access course materials, submit assignments, participate in discussions, and track their academic progress. Similarly, they enable educators to organize courses, distribute resources, monitor student performance, and conduct assessments efficiently. However, managing educational content, student data, and learning activities can be challenging without a streamlined digital system.[1]

Blackboard Learn is a commercial LMS commonly used by large universities and enterprise-level institutions. It provides advanced tools for course delivery, assessments, grading, analytics, and collaboration through discussion boards and virtual classrooms. The platform is highly scalable and secure, making it suitable for large user bases. Despite these advantages, Blackboard Learn involves high licensing and maintenance costs and offers limited flexibility for customization, making it less practical for smaller institutions with specific needs [2].

In addition to platform-based comparisons, existing literature highlights significant gaps in how LMS effectiveness is evaluated. A meta-analysis comparing LMS research in Australia and China identified a lack of standardized methodologies, inconsistent sampling techniques, and an overreliance on cross-sectional data. These limitations hinder the ability to generalize results and accurately measure long-term LMS effectiveness, emphasizing the need for more robust and longitudinal evaluation approaches [6].

Additionally, research focusing on mobile learning in developing contexts emphasizes that LMS effectiveness depends heavily on mobile readiness. Studies indicate that true accessibility requires not only responsive design but also optimized content delivery, seamless cross-device functionality, and consideration of connectivity and data-cost constraints. This reinforces the importance of building fully responsive and mobile-optimized LMS platforms [10].

2.2. Study of existing System

Moodle

Moodle is a widely used open-source Learning Management System adopted by schools, colleges, and universities. It offers a broad range of features including course creation, content management, assignment submission, quizzes, grading, attendance tracking, and discussion forums. Moodle supports plugins and themes, allowing institutions to customize functionality according to their needs. However, its setup and maintenance require strong technical knowledge, including server configuration and database management. Additionally, the user interface can be complex and less intuitive for beginners, which may increase the learning curve for both instructors and students.[1]

Blackboard Learn

Blackboard Learn is a commercial LMS primarily used by large universities and enterprise-level institutions. It provides advanced tools for course delivery, assessments, grading, analytics, and collaboration through discussion boards and virtual classrooms. The system is highly scalable and secure, making it suitable for large numbers of users. Despite its strengths, Blackboard Learn involves high licensing and maintenance costs and offers limited flexibility for customization, which can be challenging for smaller institutions with specific requirements.[2]

Coursera

Coursera is an online learning platform that partners with universities and organizations to deliver academic and professional courses. It offers structured video lectures, quizzes, peer-reviewed assignments, certificates, and progress tracking. Coursera is designed mainly for individual learners seeking skill development rather than institutional course management. As a result, educational institutions have limited control over course customization, student management, and administrative operations.[3]

Udemy

Udemy is a global online course marketplace where instructors independently create and publish courses, and learners can enroll based on interest. It supports video-based learning, downloadable resources, quizzes, and lifetime access to course content. While Udemy is flexible and accessible, it lacks formal academic structure, institutional oversight, and standardized evaluation mechanisms. It does not support role-based administrative control required by schools or universities.[4]

Canvas

Canvas is a modern, cloud-based Learning Management System known for its intuitive user interface and strong mobile compatibility. It supports course content management, assignments, assessments, grading, communication, and analytics, and integrates well with third-party educational tools. Canvas improves user experience for both instructors and students through real-time updates and seamless navigation. Despite these advantages, Canvas is a paid platform, and its subscription and implementation costs can be a limitation for small institutions or organizations with budget constraints.[5]

2.2. Literature Review

This study serves as a meta-analysis focusing on how LMS effectiveness is measured in educational research, specifically comparing approaches in Australia and China. The authors found a lack of standardized and rigorous methodologies, such as inconsistent sampling and reliance on cross-sectional data, which creates difficulties in generalizing findings. Their work emphasizes the critical need for researchers to adopt more robust and longitudinal methods to accurately assess the long-term impact and true effectiveness of LMS in diverse settings. This points to the need for your project to consider reliable evaluation metrics beyond simple course completion.[6]

Maslov et al. specifically investigated the relationship between a user's experience and their engagement with an LMS. They identified that usability, accessibility, and intuitive design are not just supplementary features, but core drivers of learner satisfaction and overall system effectiveness. The central conclusion is that a smooth, logical, and accessible interface directly reduces cognitive load for students, allowing them to focus on the content rather than the technology. This literature directly supports your project's reliance on modern frameworks (React/Tailwind) to prioritize a clean, seamless user experience.[7]

This research provides a practical, comparative analysis of different LMS platforms used in higher education institutions. The study went beyond simple feature lists to highlight how inherent differences in functionality, adaptability, and accessibility among various LMS solutions (e.g., open-source vs. commercial) directly impact teaching efficacy and student learning outcomes. Their findings underscore that the choice or design of an LMS is a pedagogical decision; a one-size-fits-all approach often fails to maximize engagement, thus justifying the need for custom, focused systems like the one you are developing.[8]

Abid and colleagues proposed a systematic evaluation framework designed to provide objective criteria for assessing and comparing LMS platforms. Their work focuses on quantitative technical aspects that are often overlooked in educational studies: usability metrics, system performance, and scalability. This framework offers a rigorous lens for institutions when selecting or improving an LMS, providing essential benchmarks for technical success. For your project, this research is crucial for defining the non-functional requirements and future performance goals of your system.[9]

This research stream, often utilizing case studies in developing contexts, examines the critical role of mobile devices in modern e-learning. Mtebe and Raisamo, among others, emphasize that achieving the "anytime, anywhere" promise of an LMS depends entirely on its mobile readiness. The study highlighted that simple responsiveness is insufficient; true effectiveness requires optimization of content, seamless cross-device functionality, and consideration of mobile data costs and connectivity challenges. This literature validates project's design choice to build a fully responsive platform, ensuring maximum accessibility and flexibility for contemporary learners.[10]

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1. System Analysis

System analysis is a critical phase in the software development lifecycle, focusing on understanding, defining, and modeling the requirements of a system to be developed or improved. The purpose of the System Analysis is to establish a comprehensive **Learning Management System (LMS)** that facilitates effective interaction between admins, educators, and students. The system is designed to provide a centralized digital learning environment where educational content can be created, managed, and accessed efficiently.

The platform features a dynamic learning interface where students can browse available courses, enroll in programs, and access learning materials such as videos, documents, and assessments. Educators can manage course content and monitor learner progress in real time.

To ensure smooth operation and structured control, the system is governed by three primary user roles:

- **System Admin**
  Holds the highest level of access to manage the entire LMS, including user accounts, role permissions, course approvals, and system configuration.
- **Educator**
  Responsible for creating and managing courses, uploading learning materials, conducting assessments, and evaluating student performance.
- **Student**
  Can register, enroll in courses, access learning resources, submit assignments, and track individual learning progress.

This system aims to improve accessibility to education, streamline course management processes, and enhance the overall learning experience through a modern web-based platform.

3.1.1. Requirement Analysis

The term requirements determination explains the overall things that can be done within the system in simpler manner. The requirements for the system can be termed as functional and non- functional requirements.

a. **Functional Requirements:**

The requirement that has been used in the project as the functional requirement generally includes the function such as inputs, the processing and the final output. The functional requirement in the project is mentioned below:

- **Register:** Students, educators, and admin can create accounts to access the system.
- **Login:** Registered users can securely log in using credentials.
- **Account Management:** Users can update their personal details, change passwords, and manage profiles.
- **Course Management:** Educators can manage courses, including creating, updating, and deleting course content.
- **Create Course:** Educators can design and add new courses with detailed information, study materials, and schedules.
- **Update Course:** Educators can modify existing courses, update materials, and manage course availability.
- **Course Enrollment:** Students can enroll in courses of interest, with enrollment records maintained by the system.
- **View Course Details:** Students and educators can view course descriptions, schedules, and materials.
- **Payment Processing:** The system supports online payment for course enrollment and generates confirmation receipts.

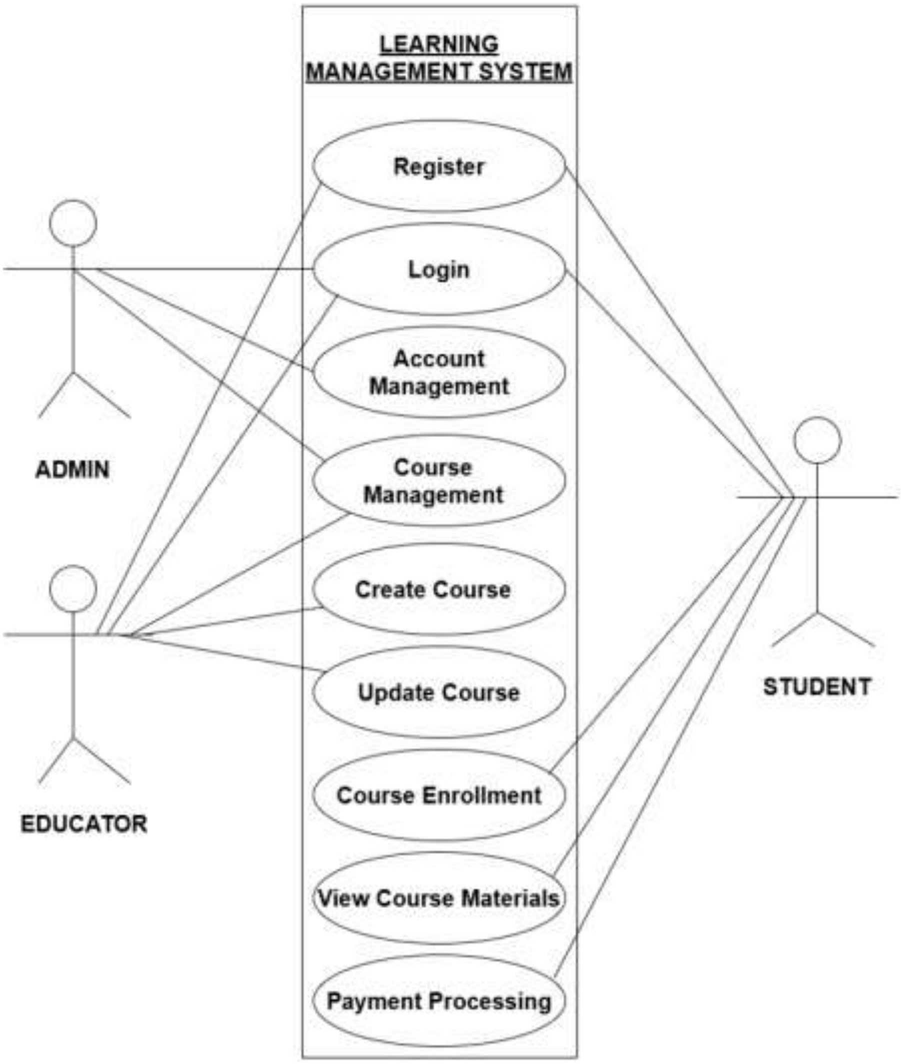


Fig 1: Use Case Diagram of Learning Management System

b. **Non-functional Requirements**

Non-functional requirement refers to criteria that describe how a system operates, rather than what is does. These are often related to the quality attributes of the system. The non-functional requirement included in the project are:

- **Effectiveness:**

  The system must effectively support teaching and learning activities by enabling students to access courses easily and allowing educators to manage learning content without complexity.

- **Usability:**
  The interface must be simple and intuitive so administrators, educators, and students can use the system easily without requiring special training.
- **Productivity:**
  The system must improve productivity by reducing manual work involved in course management, enrollment, and progress tracking for both educators and administrators.
- **Reliability:**
  The system must ensure accurate storage of user data, course information, and student progress, with proper error handling to prevent data inconsistency.

3.1.2. Feasibility Analysis

Feasibility is the examination of the system's impact on the organization during its development. This impact can be either positive or negative. If the positive outcomes outweigh the negative ones, then the system is seemed feasible. The feasibility study has been performed in the following ways:

1. Technical Feasibility

The system is technically feasible because the technology used to develop it is easily available and doesn't require expensive tools. It can be built using a simple PC with opensource software like HTML, CSS, PHP, and MySQL. These technologies are free to use, and admin can find plenty of resources and support online. This makes the system cost-effective and easy to develop, even with limited resources.

2. Operational Feasibility

The system is operationally feasible because it is easy for students to use and understand. A simple and clear interface has been provided, requiring no advanced knowledge. The system is designed to function effectively on local networks. Built with open-source tools and using MySQL for data management, smooth and reliable operation is ensured. The system is operationally feasible as it can be used easily by students with minimal training and support.

3. Economic Feasibility

The system is economically feasible because it utilizes open-source resources. Students are not required to purchase any software to access the system. From the admin perspective, it is also cost-effective as it is built with react, node.js and MySQL, which are open source. For database management, MySQL is used, which is available free of charge. This approach ensures that costs are kept low while maintaining high functionality.

4. Schedule Feasibility

The project is successfully completed within the specified time constraints and schedule. We have effectively managed our tasks and resources, ensuring that all projects milestones were achieved on time. By following the planned schedule, the project was delivered qw

Table 1: Gantt chart

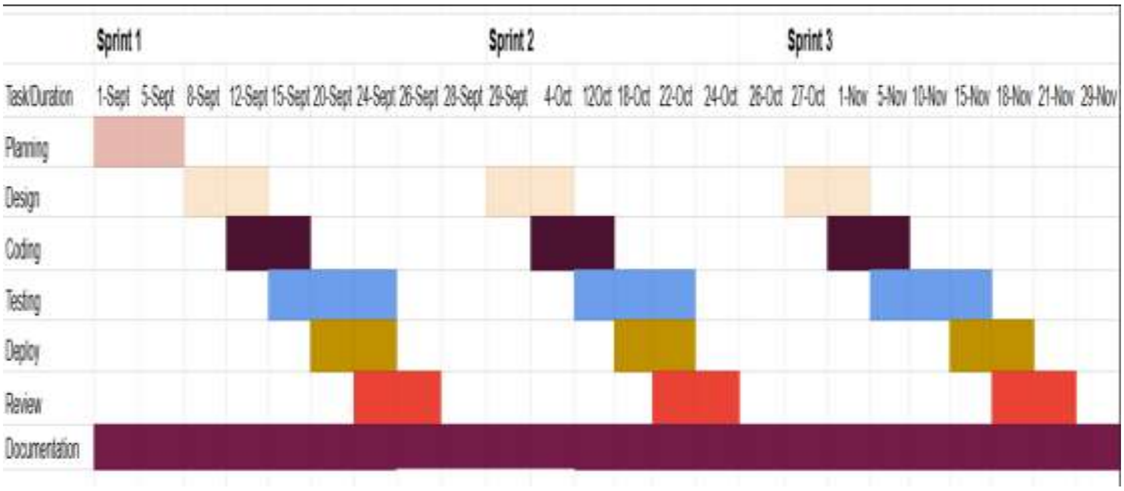


Fig 1.1: Gantt Chart of Learning Management System

3.1.3. Data Modeling (ER-Diagram)

An Entity-Relationship (ER) diagram is a visual representation of the model for a database system. This model has been used for data modeling in our project. The ER diagram of the school club management system is:

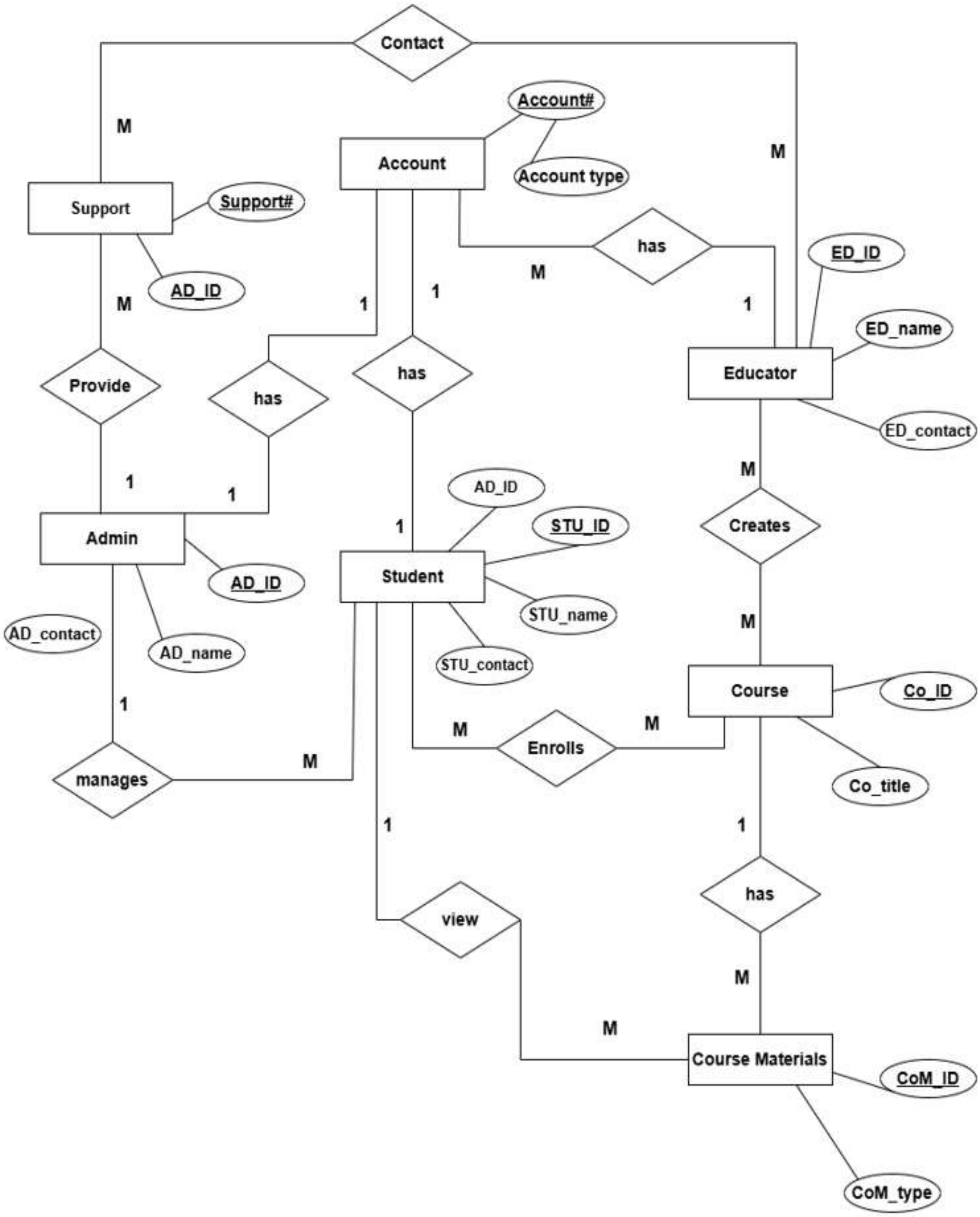


Fig 1.2: ER-Diagram of Learning Management System

3.1.4. Process Modeling (DFD):

A DFD can be referred to as a process model. DFD is a way of representing a flow of data through a process or a system.

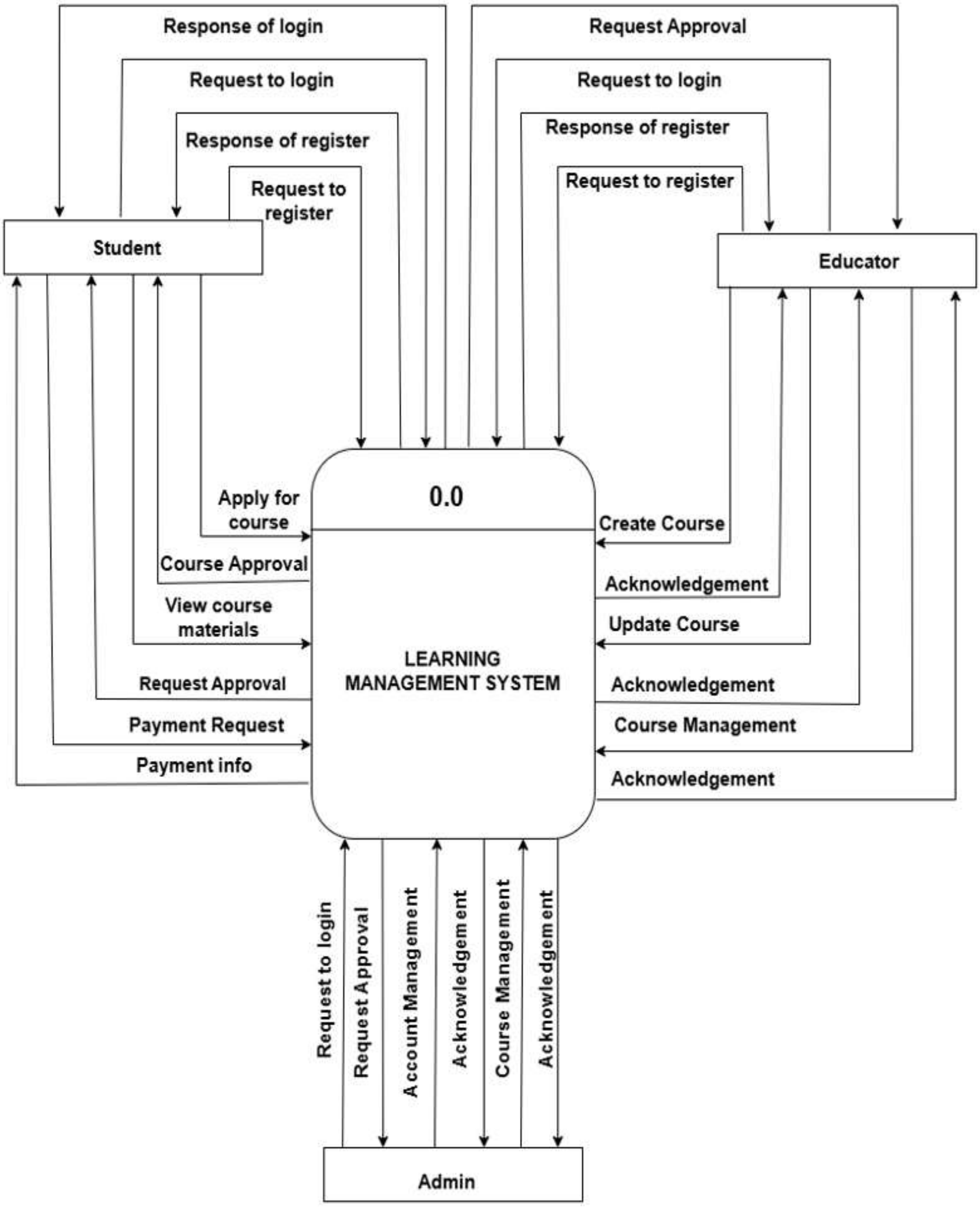


Fig 1.3: 0 Level DFD of Learning Management System

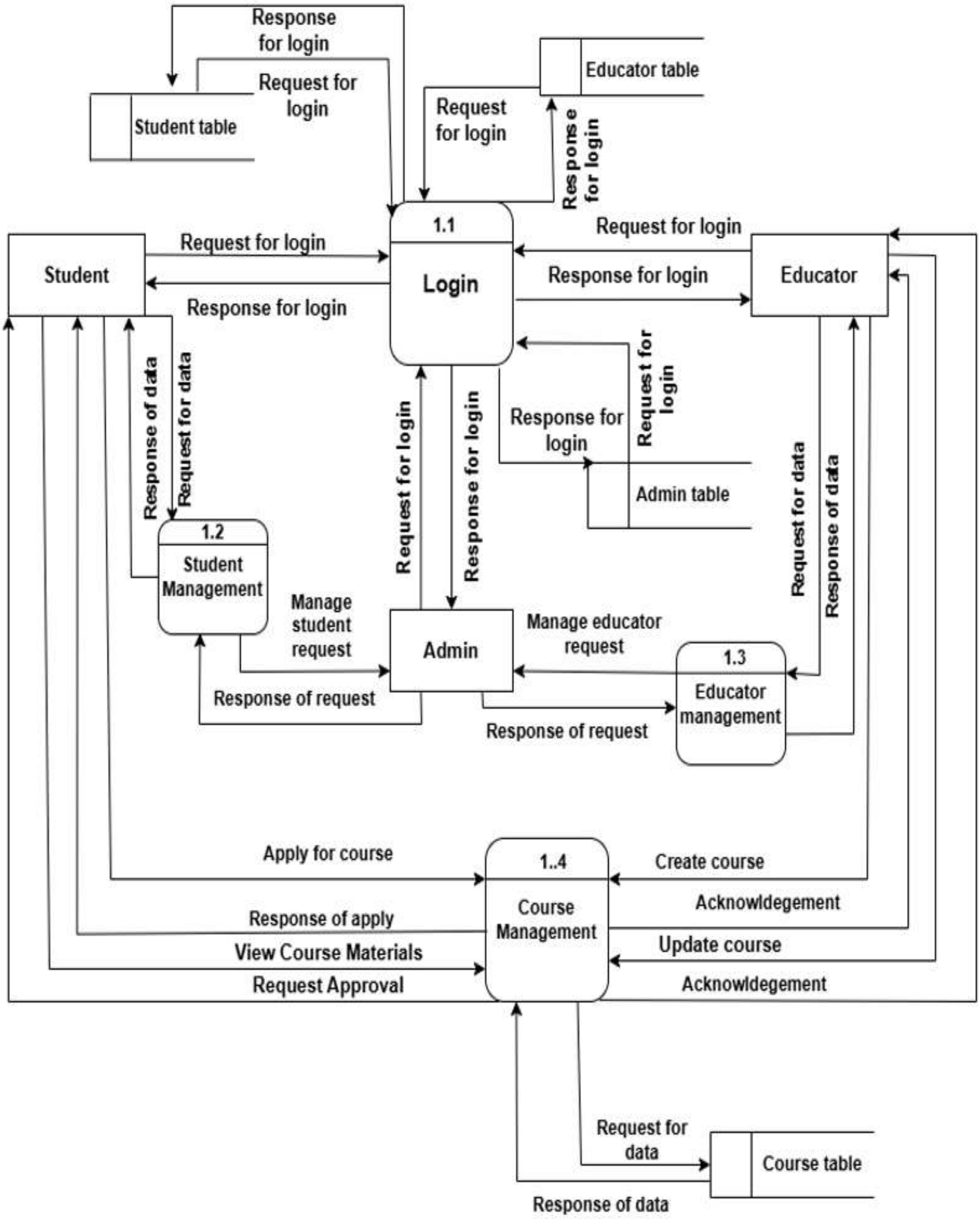


Fig 1.4: Level 1 DFD of Learning Management System

3.2. System Design

System design is the process of defining the architecture, components, and functionality of a software system. It involves creating detailed specifications, diagrams, and plans that outline how the system will be built and function. The learning management system site is designed using various schema design, CASE tools and is described using following designs:

3.2.1. Architectural Design

Architectural design is the process of figuring out the different parts that make up a system and how these parts will work together. It involves determining how these parts will communicate and be controlled to create a well-organized and efficient system.

Architectural design ensures that all the pieces fit together seamlessly to create a reliable and effective system. It serves the foundation for the development and implementation of the software.

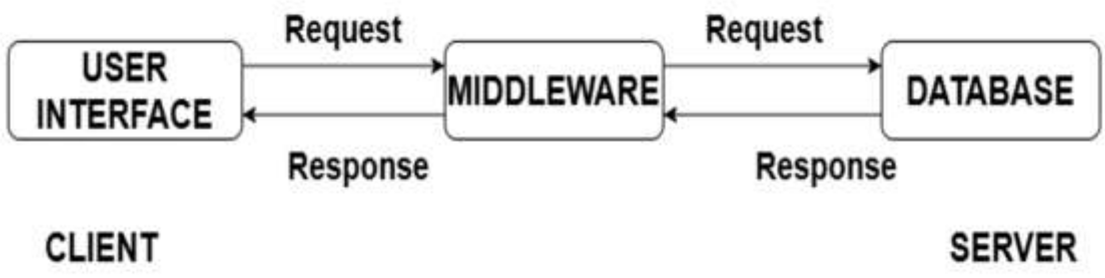


Fig 1.5: Architectural design of Learning Management System

1. User Interface Layer (React.js)

This is the front-end of the system where users interact with the LMS. It is developed using React.js and provides features like login, course browsing, and dashboards. It sends user requests to the backend and displays the responses in a user-friendly way.

2. Middleware / Application Layer (Node.js)

This layer acts as the brain of the system. Built with Node.js, it processes user requests, applies business logic, and handles authentication and validations. It communicates between the front-end and the database.

3. Database Layer (MySQL)

This is the data storage layer of the LMS. It uses MySQL to store all important information such as user details, courses, and enrollments. It responds to queries from the backend and ensures data is securely managed.

3.2.2. Database Schema Design

Database schema design organizes the data into separate entities, determines how to create relationships between organized entities, and how to apply the constraints on the data. In the figure below, there are tables in the database each of them has their own fields and identified with their respective primary key which when used in another table, becomes foreign key. The fields in each entity table have data type according to the data it stores.

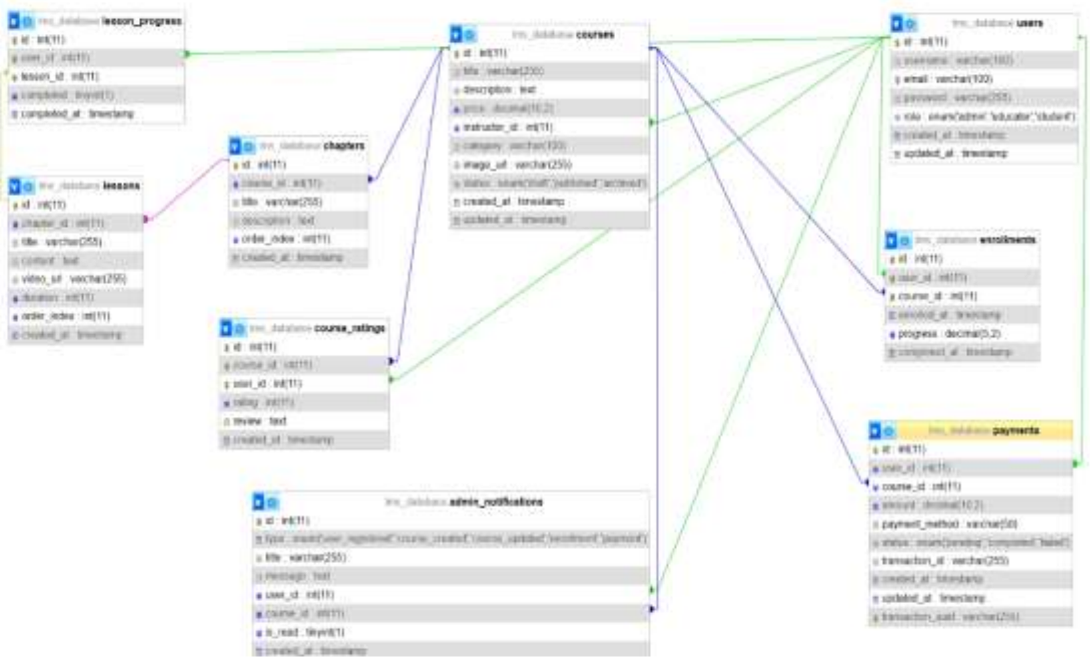


Fig 1.6: Database Schema Design of Learning Management System

3.2.3. Interface Design

User interface is the front-end application view to which user interacts in order to use the software. It is used to build interfaces in system focusing on looks and styles. Here we tried to design user friendly interfaces so that users find easy to use and pleasurable. The following are the UI design of home page, login page, user dashboard, etc.

Fig 2: Home Page of Learning Management System

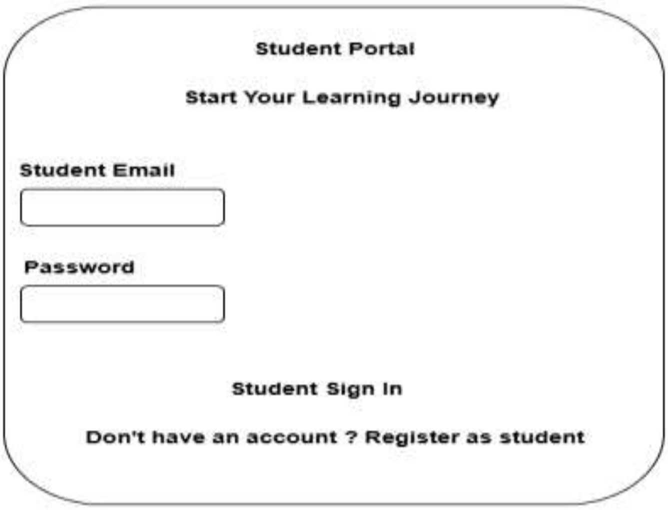


Fig 2.1: Sign in page of Student in Learning Management System

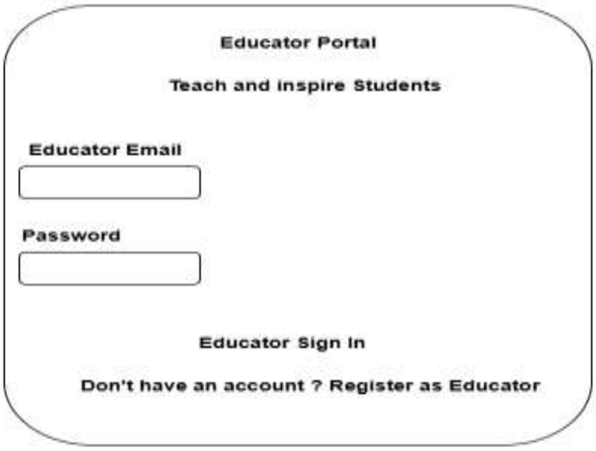


Fig 2.2: Sign in page of Educator in Learning Management System

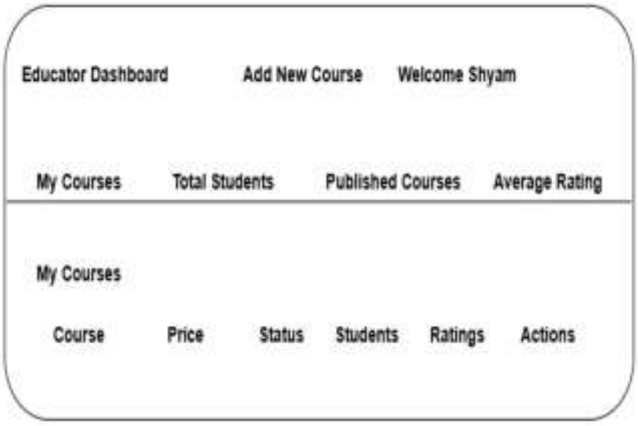


Fig 2.3: Educator Dashboard of Learning Management System

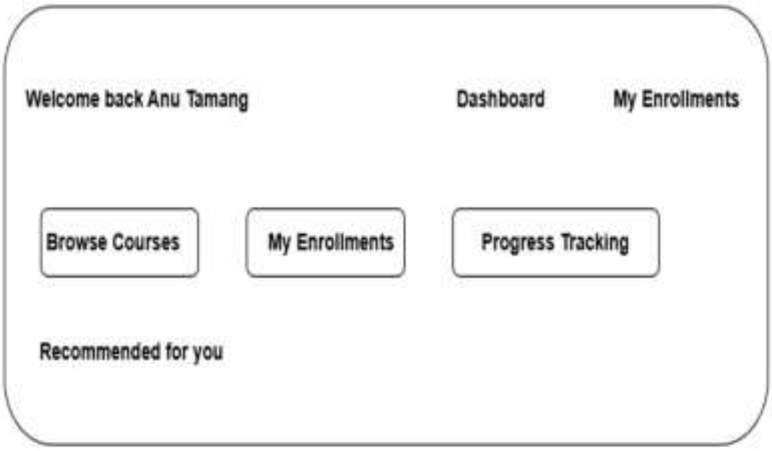


Fig 2.4: Student Dashboard of Learning Management System

3.3 High -Level Design

3.3.1 Methodology of the system

The Agile software development model is used for the development of this LMS project. The reason for choosing this model is that it is swift and versatile. Agile methods break tasks into smaller iterations, which do not require extensive long-term planning.

The project scope and requirements are laid out at the beginning of the development process. Plans regarding the number of iterations, the duration, and the scope of each iteration are clearly defined in advance. This approach allows for flexibility and adaptability, enabling developers to incorporate feedback from students, educators, and administrators and make improvements throughout the development cycle. Regular testing and continuous integration ensure that any issues are identified and resolved quickly. Agile also promotes collaboration between developers and users, ensuring that the final product meets user expectations. This iterative process helps in delivering a high-quality, efficient, and user-centric Learning Management System.

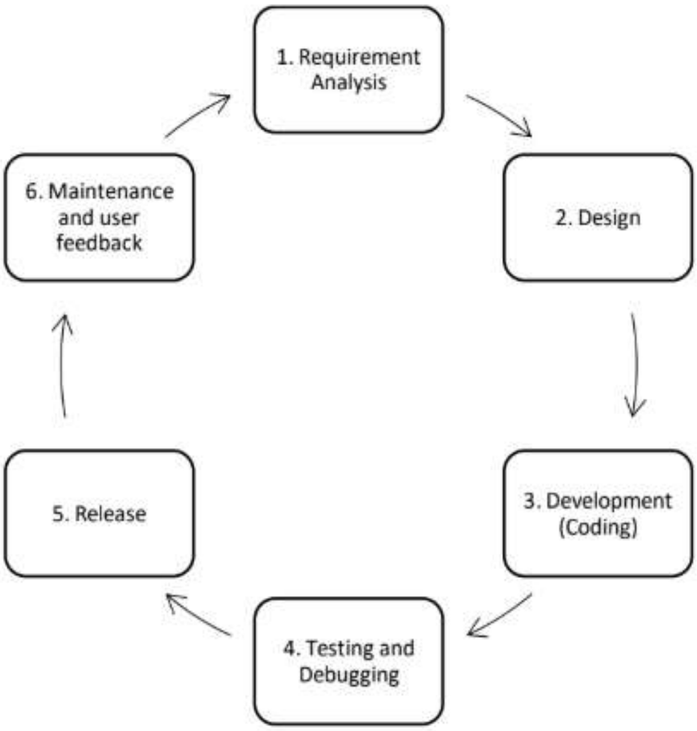


Fig 2.5: Agile Software Development Model

a) **Requirement Analysis:** Collect and analyze the needs of users to define what the system should do.
b) **Design:** Plan the system architecture, database, and user interface based on the requirements.
c) **Development (Coding):** Write the code and implement the planned features.
d) **Testing and Debugging:** Test the system to find and fix errors or issues.
e) **Release:** Deploy the system for use by students, educators, and admin.
f) **Maintenance and User Feedback:** Monitor the system, gather feedback, and make improvements for better performance and usability.

3.3.2 Algorithm Details

a. Content-Based Recommendation Algorithm

The proposed system implements a content-based recommendation approach to suggest suitable courses to learners based on their preferences and learning history. To enhance the accuracy and quality of recommendations, the Sine Cosine Algorithm (SCA) is used as an optimization technique. The algorithm searches for the best possible combination of courses by considering multiple influencing factors. The Sine Cosine Algorithm is a population-based optimization algorithm used to find the optimal set of recommended courses for a learner based on factors such as:

- Student interest score
- Previous course performance
- Course popularity
- Skill relevance

Steps of Content-Based Recommendation Algorithm

1. Initialize the dataset
2. Initialize the population
3. Initialize parameters
4. Evaluate initial fitness
5. Repeat until stopping condition is met:
    a) Select the best solution P^t (highest similarity score)

b) For each solution x_i;

i) Generate random values r_1, r_2, r_3, r_4

ii) If r_4 < 0.5, update using sine function

iii) Else, update using cosine function

iv) Ensure solution stays within valid range

v) Compute new fitness value

vi) Update best solution if improved

c) Store best solution

6. Return final recommended courses.

Mathematical Representation

$$X_i^{t+1} = \begin{Bmatrix} X_i^t + r_1 * \sin(r_2) * r_3, r_4 < 0.5 \\ X_i^t + r_1 * \cos{(r_2)} * r_3, r_4 < 0.5 \end{Bmatrix}$$

Parameter Control

The parameter r_1 decreases over time to balance exploration and exploitation:

$$r_1 = a - \frac{t * a}{T}$$

Where:

- a is a constant (usually 2)
- High r_1 → global search
- Low r_1→ local refinement

b. Fuzzy Search Algorithm

Fuzzy search algorithm is a technique used to find matches that are approximately equal rather than exactly equal. It is useful in applications where users may enter incorrect or incomplete input.

Allowed edits:

- Insertion (add a character)
- Deletion (remove a character)
- Substitution (replace a character)

Steps of Fuzzy Search Algorithm

1. Input search query

   User enters course name or keyword.

2. Load dataset

   Retrieve all course titles and related data.

3. Define allowed operations

   - Insertion
   - Deletion
   - Substitution

4. For each course in dataset:

   a) Compare query with course name

   b) Compute edit distance

   c) Calculate similarity score

   d) If similarity score is acceptable → keep the course

5. Sort results

   Arrange courses based on lowest edit distance (best match first).

6. Return matched courses to user

Mathematical Representation of Fuzzy Search Algorithm

Let:

- $\mathcal{S} = \mathcal{S}_1, \mathcal{S}_2 \ldots\ldots\ldots, \mathcal{S}_m$ (input strings)
- T= $t_1, t_2 \ldots, t_n$(course title)

Define a matrix $\mathcal{D}$ of size (m+1) × (n+1)

$$D(i,j) = \min \begin{cases} D(i-1,j)+1 & (Deletion) \\ D(i,j-1)+1 & (Insertion) \\ D(i-1,j-1)+cost & (Substitution) \end{cases}$$

Where:

$$cost = \begin{cases} 0, & if\ s_i = t_j \\ 1, & if\ s_i \neq t_j \end{cases}$$

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

In this phase, theoretical design is turned into practical working system. The most crucial stage in achieving a new successful system and in giving confidence on the system for the users that it will work efficiently and effectively. The actual implementation of the system takes place during this phase. However, before the system is fully implemented, thorough testing is done to ensure that it functions according to the specified requirements. Testing helps identify and fix any issues or problems, ensuring that the system works effectively. Only after testing and confirmation that the system meets the desired specifications, it is ready to be implemented for real-world use.

Tools Used:

Tools that are used while designing this website are:

Front End: React

React is used to design the entire user interface of the LMS. It provides a component-based architecture that improves code reusability, maintainability, and performance. React allows smooth navigation between pages without reloading the application.

Backend: Node.js and MySQL

Node.js

Node.js is a server-side runtime environment used to build the backend of the LMS. It allows JavaScript to be executed on the server and is used to handle client requests, manage APIs, and control application logic. Node.js ensures fast and scalable performance of the system.

Database tools: MySQL

MySQL

Every web application requires a database for storing collected data. MySQL which is a open source and the most popular database management system.

Diagram tools: Draw.io

Draw.io

Draw.io is a web-based diagramming tool used to design system architecture diagrams, data flow diagrams, and entity relationship diagrams (ER diagrams) for the LMS. It helps in visually representing system components and their interactions in a clear and structured manner.

4.2 Implementation Details of Modules

According to the demand, this website can be divided into several main functional modules. Basically, we have divided our project into three main functional modules.
They are:

- Student Module
- Admin Module
- Educator Module

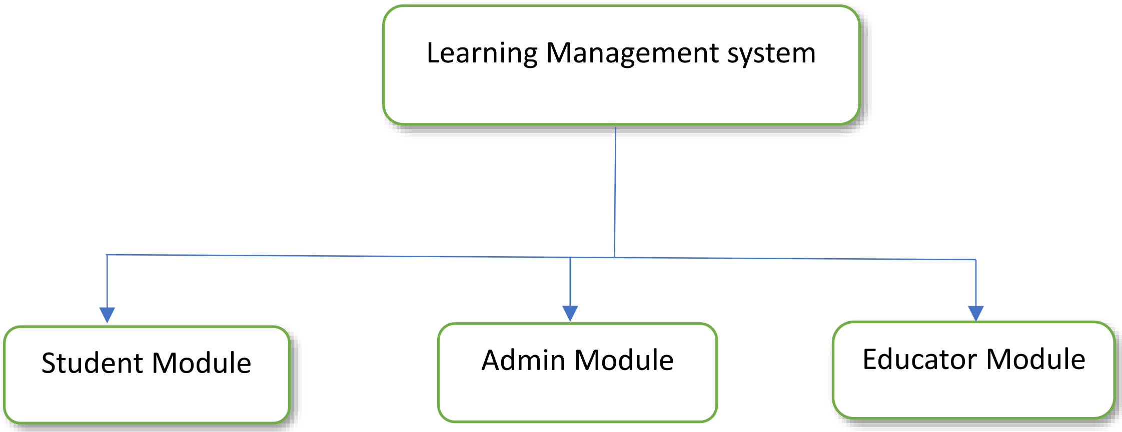


Fig 2.6: Module Implementation of Learning Management System

Student Module

The student module is designed to provide a smooth and engaging learning experience for learners. It allows students to explore available courses, enroll in subjects of interest, and interact with learning materials effectively. Students can register and log in to access the system features, view course content, submit assignments, and track their learning progress. This module focuses on delivering a user-friendly interface that supports continuous learning and academic growth.

StudentDashboard,jsx

```
import React from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
importPersonalizedRecommendationsfrom'../../components/PersonalizedRecommendatin;
```

```
const StudentDashboard = () => {
const { user } = useAuth();
 return (
    <div className="min-h-screen bg-gray-50">
      <div className="max-w-7xl mx-auto px-4 py-8">
        {/* Header */}
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-gray-900">
            Welcome back, {user?.username}!
          </h1>
          <p className="text-gray-600 mt-2">
            Continue your learning journey
          </p>
        </div>
        {/* Quick Actions */}
        <div className="grid md:grid-cols-3 gap-6 mb-8">
                      <svg
              className="w-6 h-6 text-green-600"
          {/* Progress */}
          <div className="bg-white p-6 rounded-lg shadow-md">
            <div className="flex items-center">
              <div className="w-12 h-12 bg-purple-100 rounded-lg flex items-center justify-
center">
                <svg
                  className="w-6 h-6 text-purple-600"
                  fill="none"
                  stroke="currentColor"
                  viewBox="0 0 24 24"
                >
                  <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
```

```
          strokeWidth={2}
          d="M13 10V3L4 14h7v7l9-11h-7z"
         />
        </svg>
    </div>
   </div>
  );
};
export default StudentDashboard;
```

Student.js

```
const express = require('express');
const { auth } = require('../middleware/auth');
const db = require('../config/database');
const router = express.Router();
// Enroll in course (students only)
router.post('/enroll', auth, async (req, res) => {
  try {
   // Check if user is a student
   if (req.user.role !== 'student') {
    return res
     .status(403)
     .json({ message: 'Only students can enroll in courses' });
   }
   const { courseId } = req.body;
   if (!courseId) {
    return res
     .status(400)
     .json({ message: 'Course ID is required' });
   }
   // Check if course exists
   const [courseCheck] = await db.execute(
    'SELECT id, title, price FROM courses WHERE id = ?',
```

```
    [courseId]
  );
  if (courseCheck.length === 0) {
    return res
      .status(404)
      .json({ message: 'Course not found' });
  }
  // Create admin notification
  const notificationType = course.price > 0 ? 'payment' : 'enrollment';
  const notificationTitle =
    course.price > 0 ? 'New Payment Received' : 'New Enrollment';
  const notificationMessage =
    `${req.user.username} enrolled in "${course.title}"` +
    (course.price > 0
      ? ` for $${course.price}`
      : ' (Free course)');
  try {
    await db.execute(
  const [rows] = await db.execute(`
    SELECT c.*, u.username AS instructor_name
    FROM enrollments e
    JOIN courses c ON e.course_id = c.id
  );
  res.json(rows)
  );
  res.json({ message: 'Lesson marked as completed' });
 } catch (error) {
  res
    .status(500)
    .json({ message: 'Server error', error: error.message });
 }
});
module.exports = router;
```

Admin Module

The admin module is responsible for managing and controlling the overall LMS operations. Administrators have the authority to create, update, and delete courses, manage user accounts, assign roles, and monitor system activities. They can also oversee course approvals, track student enrollment, manage feedback, and update system settings as required. This module ensures smooth functioning, data integrity, and effective system administration.

AdminDashboard.jsx

```
import React, { useState, useEffect } from 'react';
import API from '../../api/api';
import { useAuth } from '../../context/AuthContext';
const AdminDashboard = () => {
  const { user } = useAuth();
  const [students, setStudents] = useState([]);
  const [educators, setEducators] = useState([]);
  const [courses, setCourses] = useState([]);
  const [notifications, setNotifications] = useState([]);
  const [selectedStudent, setSelectedStudent] = useState(null);
  const [studentProgress, setStudentProgress] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [activeTab, setActiveTab] = useState('dashboard');
  useEffect(() => {
    if (user?.role !== 'admin') return;
    fetchData();
    const interval = setInterval(fetchData, 30000);
    return () => clearInterval(interval);
  }, [user]);
  const fetchData = async () => {
    try {
      const [
        studentsRes,
```

```
      educatorsRes,
      coursesRes,
      notificationsRes
    ] = await Promise.all([
      API.get('/admin/students'),
      API.get('/admin/educators'),
      API.get('/admin/courses'),
      API.get('/admin/notifications')
    ]);
    setStudents(studentsRes.data);
    setEducators(educatorsRes.data);
    setCourses(coursesRes.data);
    setNotifications(notificationsRes.data);
  } catch (error) {
    setError(
      error.response?.data?.message ||
      error.message ||
      'Failed to fetch data'
    });
    fetchData();
    alert('User role updated');
  } catch {
    alert('Error updating user role');
  }
};
if (loading) {
  return <div className="p-8">Loading...</div>;
}
if (error) {
  return (
    <div className="p-8 text-red-600">
      Error: {error}
    </div>
  );
```

```
<th className="p-3">Email</th>
                <th className="p-3">Courses</th>
                <th className="p-3">Spent</th>
                <th className="p-3">Actions</th>
              </tr>
            </thead>
            <tbody>
              {students.map(s => (
                <tr key={s.id} className="border-t">
                  <td className="p-3">{s.username}</td>
                  <td className="p-3">{s.email}</td>
                  <td className="p-3 text-center">{s.enrolled_courses}</td>
                  <td className="p-3 text-green-600">
                    NRS {Number(s.total_spent || 0).toFixed(2)}
                  </td>
        )}
      </div>
    </div>
  );
};
export default AdminDashboard;
```

Admin.js

```
const express = require('express');
const { auth, adminAuth } = require('../middleware/auth');
const db = require('../config/database');
const router = express.Router();
// Get dashboard stats
router.get('/dashboard', auth, adminAuth, async (req, res) => {
  try {
    const [users] = await db.execute('SELECT role, COUNT(*) as count FROM users GROUP BY role');
    const [courses] = await db.execute('SELECT status, COUNT(*) as count FROM courses GROUP BY status');
```

```
    const [enrollments] = await db.execute('SELECT COUNT(*) as total FROM enrollments');
      WHERE u.role = 'educator'
      GROUP BY u.id
      ORDER BY u.created_at DESC
    `);
});
// Get student progress details
router.get('/student/:id/progress', auth, adminAuth, async (req, res) => {
  try {
    const [progress] = await db.execute(`
      SELECT c.title as course_title, c.price, e.enrolled_at, e.progress,
             COUNT(DISTINCT l.id) as total_lessons,
             COUNT(DISTINCT lp.lesson_id) as completed_lessons
      FROM enrollments e
      JOIN courses c ON e.course_id = c.id
      WHERE e.user_id = ?
      GROUP BY c.id
      ORDER BY e.enrolled_at DESC
    `, [req.params.id]);
    res.json(progress);
  } catch (error) {
    res.status(500).json({ message: 'Server error', error: error.message });
  }
});
router.put('/user-role', auth, adminAuth, async (req, res) => {
  try {
    const { user_id, role } = req.body;
    await db.execute('UPDATE users SET role = ? WHERE id = ?', [role, user_id]);
    res.json({ message: 'User role updated successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Server error', error: error.message });
  }});
module.exports = router;
```

Educator Module

The educator module enables instructors to create and manage course content, upload learning materials, design assessments, and evaluate student performance. Educators can interact with students through assignments and feedback, ensuring effective knowledge delivery.

EducatorDashboard.jsx

```
import React, { useState, useEffect } from 'react';
import { useAuth } from '../../context/AuthContext';
import { Link } from 'react-router-dom';
import API from '../../api/api';
const EducatorDashboard = () => {
  const [courses, setCourses] = useState([]);
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);
  const [showStudents, setShowStudents] = useState(false);
  const [allStudents, setAllStudents] = useState([]);
  const { user, logout } = useAuth();
  useEffect(() => {
    fetchEducatorData();
  }, [user]);
  const fetchEducatorData = async () => {
    try {
      const response = await API.get('/educator/courses');
      console.log('Educator courses data:', JSON.stringify(response.data, null, 2));
      setCourses(response.data);
    } catch (error) {
      console.error('Error fetching educator data:', error);
      setCourses([]);
    } finally {
      setLoading(false);
    }
  };
```

```
 const fetchAllStudents = async () => {
   try {
     console.log('Fetching students...');
     const response = await API.get('/educator/students');
     console.log('Students data:', response.data);
     setAllStudents(response.data);
     setShowStudents(true);
   } catch (error) {
     console.error('Error fetching students:', error);
     alert('Error loading students: ' + error.message);
   }
 };
          </p>
        </div>
        <div
          className="bg-white p-6 rounded-lg shadow cursor-pointer hover:shadow-lg
transition"
          onClick={() =>
            document.querySelector('.courses-table')?.scrollIntoView({ behavior: 'smooth' })
          }
          className="bg-white p-6 rounded-lg shadow cursor-pointer hover:shadow-lg
transition"
          onClick={() =>
            document.querySelector('.courses-table')?.scrollIntoView({ behavior: 'smooth' })
          }
          <h3 className="text-lg font-semibold text-gray-900">Average Rating</h3>
          <p className="text-3xl font-bold text-yellow-600">
            {courses.length > 0
              ? (
                  courses.reduce(
                    (total, course) => total + (parseFloat(course.avg_rating) || 0),
                    0
                  ) / courses.length
                ).toFixed(1)
```

```
          : '0.0'}
        </p>
      </div>
    </div>
    {/* Courses List */}
          Create Your First Course
        </Link>
      </div>
    ) : (
      <div className="overflow-x-auto">
        <table className="min-w-full divide-y divide-gray-200">
          <thead className="bg-gray-50">
          </thead>
          <tbody className="divide-y divide-gray-200">
            {courses.map(course => (
              <tr key={course.id}>
                <td className="px-6 py-4">
                  <div className="font-medium text-gray-900">{course.title}</div>
                  <div className="text-sm text-gray-600">{course.category}</div>
                </td>
                <td className="px-6 py-4">
                  {course.price === 0 ? (
                    <span className="text-green-600 font-semibold">Free</span>
                  ) : (
                    <span className="font-semibold">रु{course.price}</span>
                  )}
                </td>
                <td className="px-6 py-4">
                  <span className={`px-2 py-1 rounded-full text-xs font-semibold ${
                    course.status === 'published'
                      ? 'bg-green-100 text-green-800'
  </div>
  {/* Students Modal */}
```

```
              {allStudents.map((student, index) => (
               <tr key={index}>
                <td className="px-6 py-4">
                 <div className="font-medium">{student.student_name}</div>
                 <div className="text-sm">{student.student_email}</div>
                </td>
                <td className="px-6 py-4">
                 <div className="font-medium">{student.course_title}</div>
                 <div className="text-sm">{student.course_category}</div>
                </td>
                <td className="px-6 py-4">
                 {new Date(student.enrolled_at).toLocaleDateString()}
                </td>
               </tr>
              ))}
             </tbody>
            </table>
           )}
          </div>
         </div>
        )}
       </div>
  );
};
export default EducatorDashboard;
```

Educator.js

```
const express = require('express');
const { auth, educatorAuth } = require('../middleware/auth');
const Course = require('../models/Course');
const db = require('../config/database');
const router = express.Router();
// Get educator's courses
router.get('/courses', auth, educatorAuth, async (req, res) => {
```

```
  try {
    console.log('=== EDUCATOR COURSES DEBUG ===');
    console.log('Educator ID:', req.user.id);

    // Test basic query first
    const [basicRows] = await db.execute('SELECT * FROM courses WHERE instructor_id = ?', [req.user.id]);
    console.log('Basic courses:', basicRows);
    // Test enrollment count query
    const [enrollmentRows] = await db.execute('SELECT course_id, COUNT(*) as count FROM enrollments WHERE course_id IN (SELECT id FROM courses WHERE instructor_id = ?) GROUP BY course_id', [req.user.id]);
    console.log('Enrollment counts:', enrollmentRows);
    // Full query with joins
    const [rows] = await db.execute(`
      SELECT c.*,
             COUNT(e.id) as enrolled_count,
             COALESCE(AVG(r.rating), 0) as avg_rating
      FROM courses c
      LEFT JOIN enrollments e ON c.id = e.course_id
      LEFT JOIN course_ratings r ON c.id = r.course_id
      WHERE c.instructor_id = ?
      GROUP BY c.id
      ORDER BY c.created_at DESC
    `, [req.user.id]);
    console.log('Final result with counts:', rows);
    res.json(rows);
  } catch (error) {
    console.error('Error fetching educator courses:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
});
// Get all students enrolled in educator's courses
router.get('/students', auth, educatorAuth, async (req, res) => {
```

```
  try {
    const [rows] = await db.execute(`
      SELECT u.username as student_name, u.email as student_email,
             c.title as course_title, c.category as course_category,   e.enrolled_at
      FROM enrollments e
      JOIN users u ON e.user_id = u.id
      JOIN courses c ON e.course_id = c.id
      WHERE c.instructor_id = ?
      ORDER BY e.enrolled_at DESC
    `, [req.user.id]);
    res.json(rows);
  } catch (error) {
    console.error('Error fetching students:', error);
    res.status(500).json({ message: 'Server error', error: error.message });
  }
});
module.exports = router;
```

4.3 Testing

Testing is a process of executing a program with the intent of finding an error. It is the process of trying something to see if it works as expected. The primary objective for test case design is to derive a set of tests that has the highest livelihood for uncovering defects in software.

To accomplish this objective two different categories of test case design are used.

4.4. Test Case for Unit Testing

Unit testing emphasizes the verification effort on the smallest unit of software design i.e., a software component or module. Unit testing is performed parallel with the coding phase. It tests unit or module not the whole software. We have tested each view/module of the applications individually. As the modules were built up testing was carried out simultaneously, tracking out each and every kind of input and checking the corresponding output until module is working correctly.

Table 2: Unit testing for Student Registration

Test ID	Test Scenario	Test Input				Expected output	Actual Output	Status
		Name	Email	Password	Cpassword			
T1	Sign-in	Goma Magar	goma12 @ gmail.co m	goma@123	goma@123	Registered successfully	Pass	Success
T2	Sign-in	Anu Tamang	anu12@g mail.com	anu@123	anu@123	Registered Successfully	Pass	Success

Table 3: Unit testing for Student Login

Test ID	Test Scenario	Test Input		Expected Output	Actual Output	Status
		Email	Password			
T3	Login	goma12 @	goma@123	Login Successfully	Pass	Success
T4	Login	anu12@ gmail	anu@123	Login Successfully	Pass	Success

Table 4: Unit Testing for Educator Registration

Test ID	Test Scenario	Test Input				Expected Output	Actual Output	Status
		Name	Email	Password	Cpassword			
T5	Sign-in	Ram	ram1@gmail .com	ram@123	ram@123	pass	pass	success
T6	Sign-in	Shyam	Shyam1@gm ail.com	shyam@12	shyam@12	pass	pass	success

Table 5: Unit Testing for Educator Login

Test ID	Test Scenario	Test Input		Expected Output	Actual Output	Status
		Email	**Password**			
T7	Login	ram1@gmail .com	ram@123	Login Successfully	pass	Success
T8	Login	Shyam1@gmail.com	shyam@12	Login Successfully	pass	success

Table 6: Unit Testing for Admin Login

Test ID	Test Scenario	Test Input		Expected output	Actual Output	Status
		Email	**Password**			
T9	Login	admin@lms.com	password	pass	pass	Success

Table 7: Unit Testing for Course adding

Test ID	Test Scenario	Test Steps	Test data	Expected Output	Status
T10	Add Course	Go to the course and click in add details	Fill every course in details	Courses added successfully	Success

4.5 Test Cases for System Testing

System testing tests the integration of each module in the system. We have tested whether the entire system is working properly or not. Different verifications and validations are done by giving input values to the system and by comparing with expected output.

Table 8: System testing of Learning Management System

Test id	Test Scenario	Test data	Expected result	Actual Result	Status
T1	Sign-in	Name, Email, Password, Cpassword	Registered Successfully	Pass	Success
T3	Login	Email, Password	Login Successfully	Pass	Success
T5	Sign-in	Name, Email, Password, Cpassword	Registered Successfully	Pass	Success
T7	Login	Email, Password	Login Successfully	Pass	Success
T9	Login	Email, Password	Login Successfully	Pass	Success
T10	Add Course	Fill every course in details	Courses added successfully	Pass	Success

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION

5.1. Lesson Learnt/Outcome

- **Modern Frontend Development:**
  Successfully learned React.js to build a responsive and interactive learning interface, providing a smooth user experience across desktops and mobile devices while improving development efficiency through reusable components.
- **Efficient backend Integration:**
  Gained strong practical experience with Node.js and RESTful APIs to handle user authentication, course management, and data exchange between the frontend and backend reliably.
- **Personalized Learning Experience:**
  Implemented a content-based recommendation system using cosine similarity, enabling students to receive course suggestions based on their interests and learning history.
- **Secure Role-Based Access:**
  Applied JWT authentication and middleware to ensure secure, role-based access for Admins, Educators, and Students, allowing each role to access only authorized features and data.
- **Modular System Architecture:**
  Designed the LMS using a modular and layered architecture, separating frontend, backend logic, and database, which improves maintainability and scalability.
- **Production-Ready Educational Platform:**
  Delivered a fully functional LMS that demonstrates the effectiveness of the React–Node.js stack in developing modern, scalable, and user-centric web-based educational systems.

5.2 Conclusion

The Learning Management System (LMS) project has been successfully developed as a web-based platform to support online learning activities for students and educators. The primary objective of the project was to create a user-friendly system that simplifies course management, student enrollment, and progress tracking. These objectives have been successfully achieved through the implementation of an interactive and efficient system.

The use of modern frontend technologies such as React.js, along with backend tools like Node.js, and MySQL, has resulted in a responsive and reliable application. The system provides easy access to courses, learning materials, and dashboards, allowing users to interact with the platform smoothly.

In conclusion, the integration of frontend and backend technologies has produced an efficient, scalable, and user-friendly LMS. The system fulfills its intended purpose by enhancing the learning experience and simplifying the management of educational content and user data.

5.3. Future Recommendation

The LMS project has been designed in a modular and flexible manner, allowing it to accommodate future modifications and enhancements. The system architecture supports scalability, making it easier to add new features as requirements evolve.

The following enhancements can be considered as future scope of the project:

a) AI based Recommendation System

The current content-based recommendation algorithm can be enhanced by integrating AI and machine learning techniques to provide more accurate and adaptive course suggestions based on user behavior and performance.

b) Live Classes and Video Conferencing

The LMS can be extended to support live online classes using video conferencing tools, enabling real-time interaction between educators and students.

c) Admin-Mediated Online Payments for Educators

The system can be upgraded to include a secure online payment module where students pay the platform (admin), and the admin distributes payments to educators. This ensures secure transactions, proper tracking, and easier management of commissions, refunds, and financial records. Integration with popular payment gateways (such as E-Sewa , Khalti or local banking APIs) can facilitate smooth and reliable payments.

REFERENCES

[1] Moodle, “Moodle – Open-source learning platform,” *Moodle Pty Ltd.*, Perth, Australia. [Online]. Available: https://moodle.org. Accessed: Jan. 2026.

[2] Blackboard, “Blackboard Learn,” *Blackboard Inc.*, Washington, DC, USA. [Online]. Available: https://www.blackboard.com. Accessed: Jan. 2026.

[3] Coursera, “Coursera: Online courses from top universities,” *Coursera Inc.*, Mountain View, CA, USA. [Online]. Available: https://www.coursera.org. Accessed: Jan. 2026.

[4] Udemy, “Udemy online learning platform,” *Udemy Inc.*, San Francisco, CA, USA. [Online]. Available: https://www.udemy.com. Accessed: Jan. 2026.

[5] Instructure, “Canvas Learning Management System,” *Instructure Inc.*, Salt Lake City, UT, USA. [Online]. Available: https://www.instructure.com/canvas. Accessed: Jan. 2026.

[6] T. S. C. Teo, S. L. Lim, and J. Y. Lai, “A meta-analysis of learning management system success: Comparisons between Australia and China,” *Computers & Education*, vol. 168, pp. 104–118, 2021.

[7] I. Maslov, A. Nikou, and A. Hansen, “User experience and engagement in learning management systems,” *Education and Information Technologies*, vol. 26, no. 2, pp. 1953–1972, 2021.

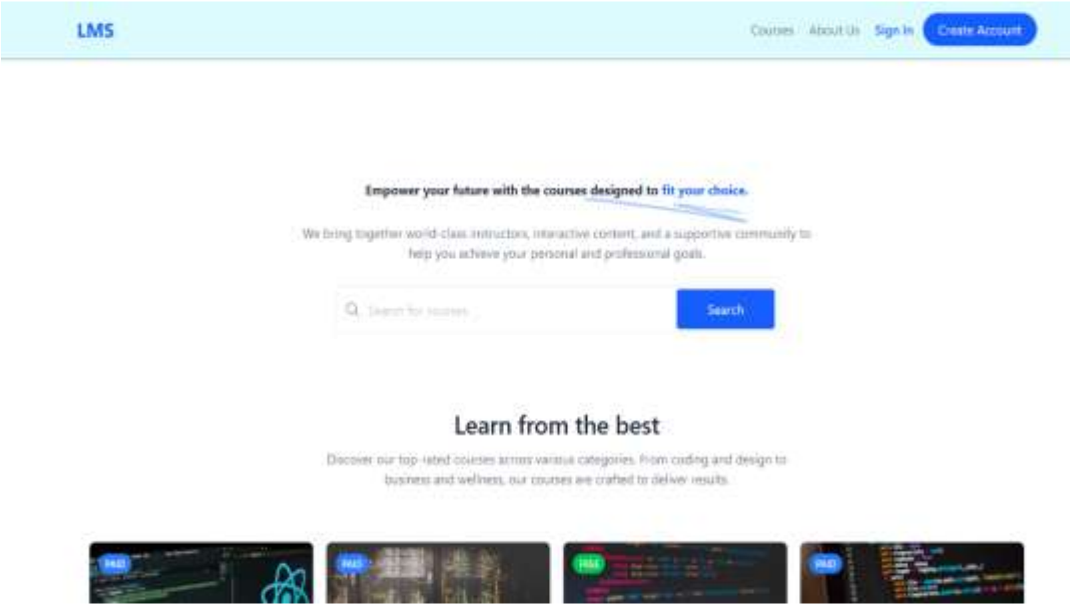
[8] A. Alshammari, A. Alhassan, and S. Aljohani, “A comparative study of learning management systems in higher education,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 4, pp. 45–52, 2021.

[9] A. Abid, M. A. Khan, and S. Hussain, “A framework for evaluating learning management systems based on usability and performance,” *Journal of Educational Technology Systems*, vol. 49, no. 3, pp. 305–322, 2020.

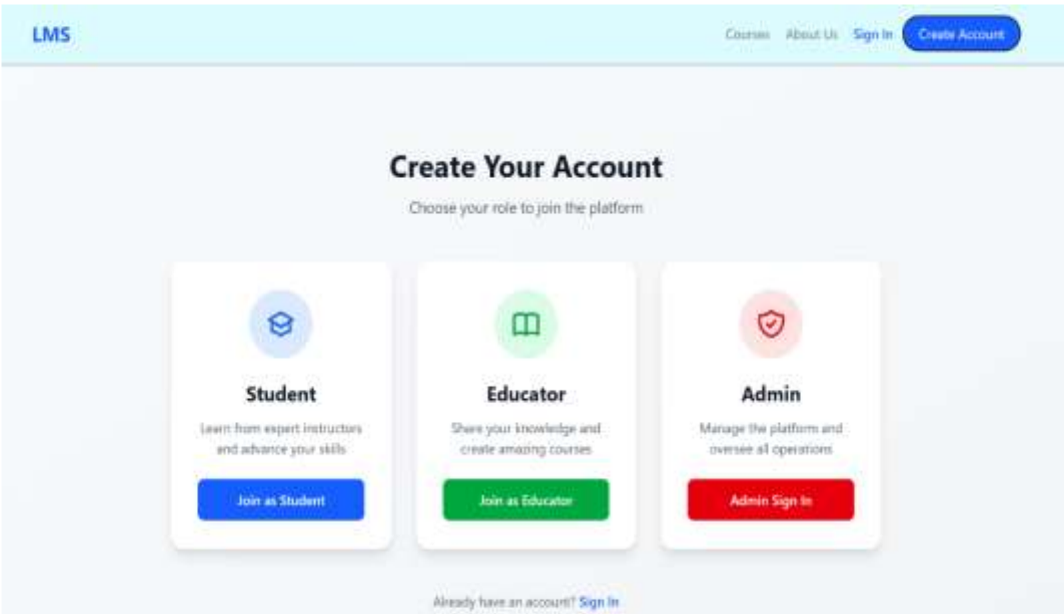
[10] J. S. Mtebe and R. Raisamo, “Challenges and opportunities of mobile learning in higher education institutions,” *International Journal of Education and Development using Information and Communication Technology*, vol. 10, no. 1, pp. 45–63, 2014.

APPENDICES

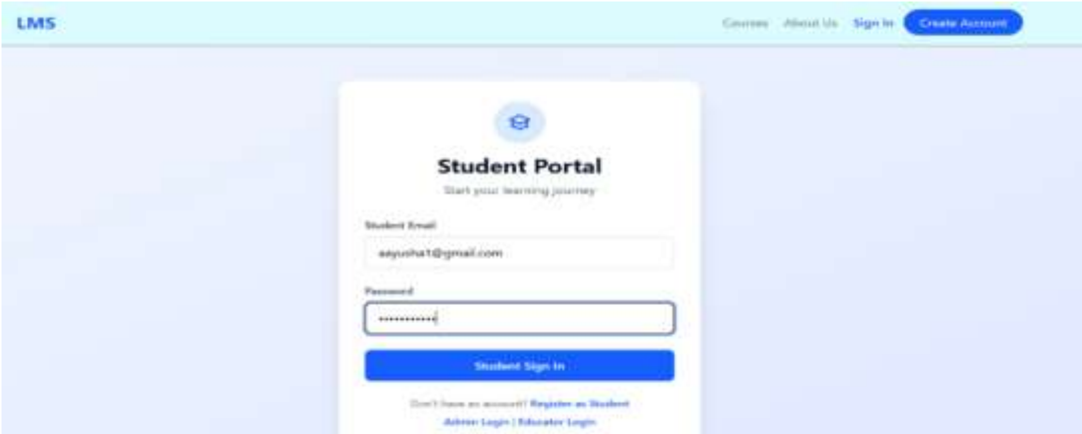
Home page



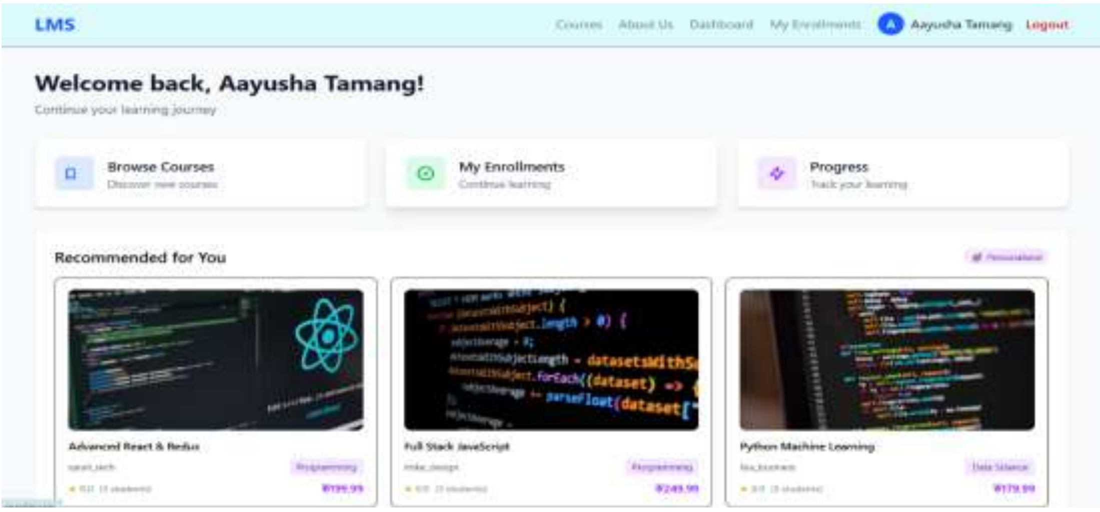
Accounts



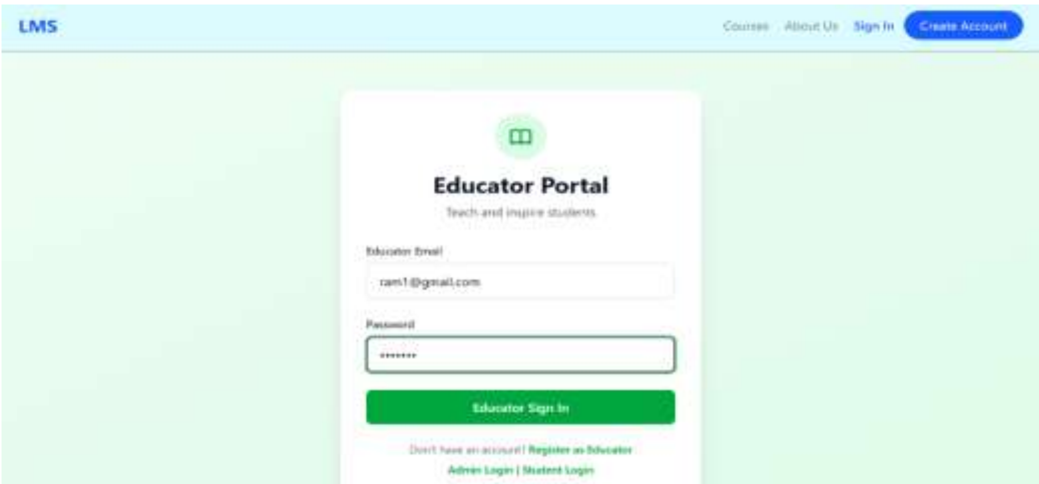
Student Login Page



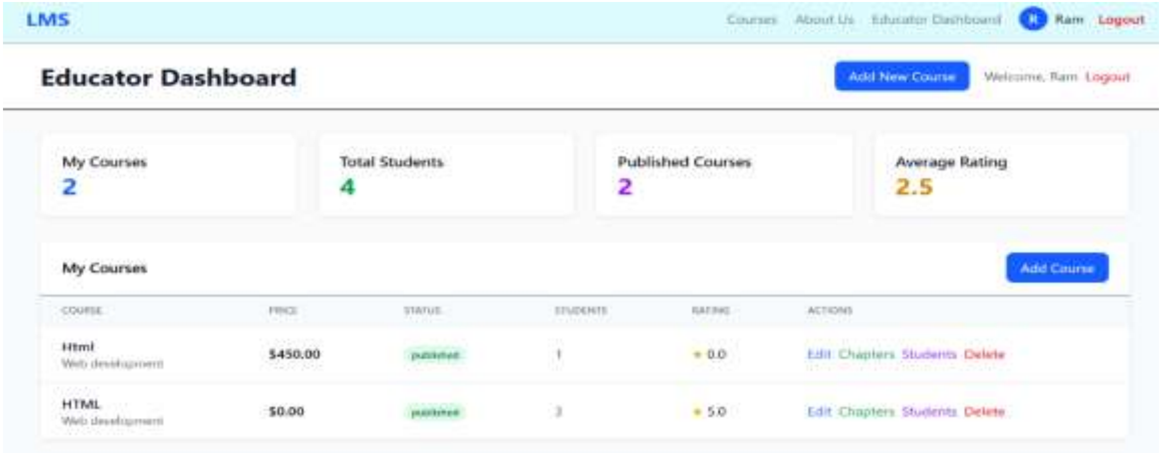
Student Dashboard Page



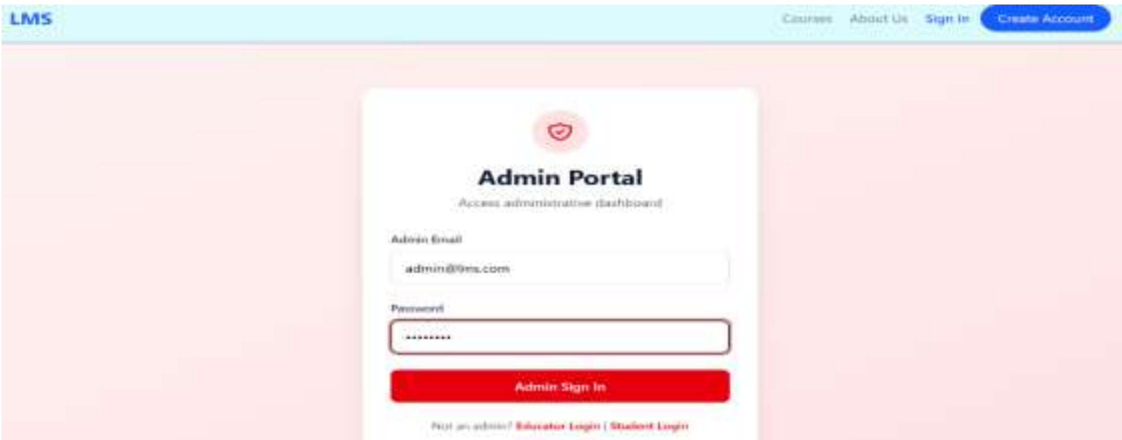
Educator Login Page



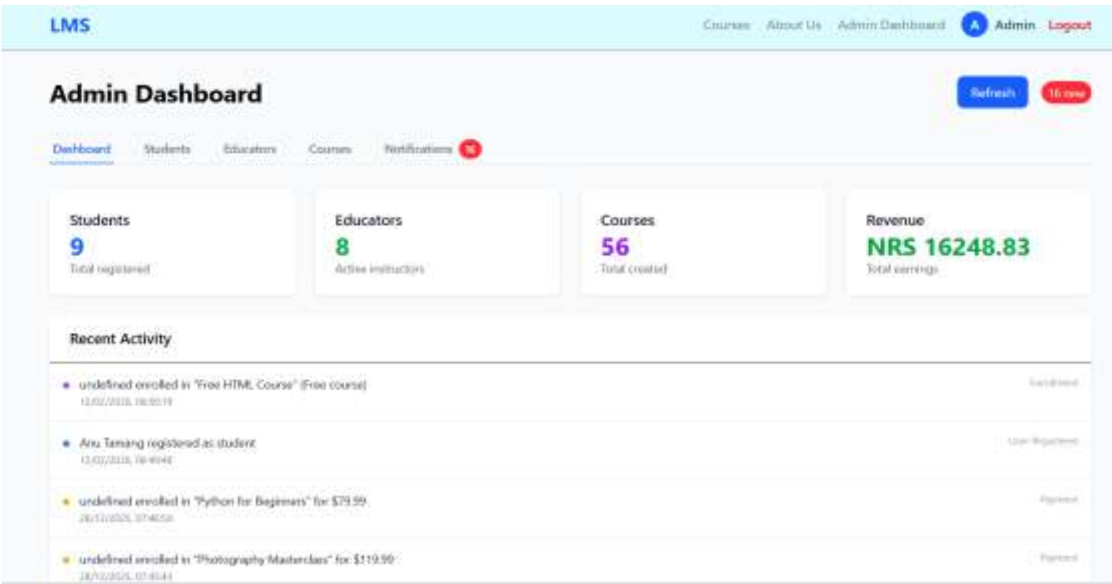
Educator Dashboard Page



Admin Login Page



Admin Dashboard Page



Esewa Payment Integration Page